

```
1 import json
2 from collections import defaultdict
3 from kafka import KafkaConsumer
4
5 consumer = KafkaConsumer(
6     "store_events",
7     bootstrap_servers="localhost:9092",
8     value_deserializer=lambda v: json.loads(v.decode("utf-8")),
9     auto_offset_reset="earliest",
10    enable_auto_commit=True,
11    group_id="store-consumers"
12 )
13
14 visit_counts = defaultdict(int)
15 active_sessions = defaultdict(list)
16 coupon_redemptions = defaultdict(int)
17
18 print("Consumer started. Listening on 'store_events'...")
19
20 for msg in consumer:
21     event = msg.value
22     et = event.get("event_type")
23
24     # Customer behavior: track sessions
25     if et == "aisle_visit":
26         customer_id = event["customer_id"]
27         aisle = event["aisle"]
28         if aisle not in active_sessions[customer_id]:
29             active_sessions[customer_id].append(aisle)
30         print(json.dumps({
31             "event_type": "aisle_visit",
32             "customer_id": customer_id,
33             "aisle": aisle,
```

```

34         "current_session_aisles": active_sessions[customer_id],
35         "timestamp": event["timestamp"]
36     }, indent=2))
37
38 elif et == "visit_end":
39     customer_id = event["customer_id"]
40     visit_counts[customer_id] += 1
41     print(json.dumps({
42         "event_type": "visit_end",
43         "customer_id": customer_id,
44         "visit_number": visit_counts[customer_id],
45         "aisles_this_trip": active_sessions[customer_id],
46         "start_timestamp": event.get("start_timestamp"),
47         "end_timestamp": event.get("end_timestamp")
48     }, indent=2))
49     print() # blank line for readability
50     active_sessions[customer_id] = []
51
52 # Transactions
53 elif et == "transaction":
54     print(json.dumps({
55         "event_type": "transaction",
56         "transaction_id": event["transaction_id"],
57         "customer_id": event["customer_id"],
58         "amount": event["amount"],
59         "timestamp": event["timestamp"]
60     }, indent=2))
61
62 # Inventory alerts
63 elif et == "inventory":
64     if event["alert_type"] in ("low_stock", "expiry"):
65         print(json.dumps({
66             "event_type": "inventory_alert",

```

```
67         "item": event["item"],
68         "alert_type": event["alert_type"],
69         "timestamp": event["timestamp"]
70     }, indent=2))
71
72     # Promotions
73     elif et == "promotion":
74         campaign = event["campaign_id"]
75         coupon_redemptions[campaign] += 1
76         print(json.dumps({
77             "event_type": "promotion",
78             "campaign_id": campaign,
79             "total_redemptions": coupon_redemptions[campaign],
80             "timestamp": event["timestamp"]
81         }, indent=2))
```

```
1 import json
2 import random
3 import time
4 from datetime import datetime
5 from kafka import KafkaProducer
6
7 producer = KafkaProducer(
8     bootstrap_servers="localhost:9092",
9     value_serializer=lambda v: json.dumps(v).encode("utf-8")
10 )
11
12 aisles = ["dairy", "meat", "produce", "bakery", "beverages", "snacks"]
13 campaigns = ["CAMP1", "CAMP2"]
14
15 def now_ms():
16     return datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
17
18 def produce_customer_behavior():
19     customer_id = f"C{random.randint(1000, 9999)}"
20     visit_number = random.randint(1, 5)
21     num_aisles = random.randint(2, 5)
22     aisles_this_trip = random.sample(aisles, num_aisles)
23     start_ts = now_ms()
24
```

```
25     # Send aisle_visit events
26     current_session = []
27     for aisle in aisles_this_trip:
28         current_session.append(aisle)
29         event = {
30             "event_type": "aisle_visit",
31             "timestamp": now_ms(),
32             "customer_id": customer_id,
33             "aisle": aisle,
34             "current_session_aisles": current_session.copy()
35         }
36         producer.send("store_events", event)
37         print(json.dumps(event, indent=2))
38         time.sleep(random.uniform(0.05, 0.2))
39
40     # Send visit_end summary
41     visit_end_event = {
42         "event_type": "visit_end",
43         "customer_id": customer_id,
44         "visit_number": visit_number,
45         "start_timestamp": start_ts,
46         "end_timestamp": now_ms(),
47         "aisles_this_trip": aisles_this_trip
```

```
48     }
49     producer.send("store_events", visit_end_event)
50     print(json.dumps(visit_end_event, indent=2))
51     print() # blank line for readability
52
53 # Main loop
54 event_types = ["transaction", "inventory", "customer_behavior", "promotion"]
55
56 while True:
57     et = random.choice(event_types)
58
59     if et == "transaction":
60         event = {
61             "event_type": "transaction",
62             "transaction_id": random.randint(10000, 99999),
63             "customer_id": f"C{random.randint(1000, 9999)}",
64             "amount": round(random.uniform(5, 100), 2),
65             "timestamp": now_ms()
66         }
67         producer.send("store_events", event)
68         print(json.dumps(event, indent=2))
69
```

```

70     elif et == "inventory":
71         event = {
72             "event_type": "inventory",
73             "item": random.choice(aisles),
74             "alert_type": random.choice(["normal", "low_stock", "expiry"]),
75             "timestamp": now_ms()
76         }
77         producer.send("store_events", event)
78         print(json.dumps(event, indent=2))
79
80     elif et == "customer_behavior":
81         produce_customer_behavior()
82
83     else: # promotion
84         event = {
85             "event_type": "promotion",
86             "campaign_id": random.choice(campaigns),
87             "timestamp": now_ms()
88         }
89         producer.send("store_events", event)
90         print(json.dumps(event, indent=2))
91
92     time.sleep(1)

```

Solution 3a)

-- Load participant information

```
participants = LOAD 'hdfs://path/to/participants.csv' USING PigStorage(',')

```

```
AS (runner_id:int, name:chararray, age:int, gender:chararray);

```

-- Load hydration station logs

```
hydration = LOAD 'hdfs://path/to/hydration_logs.csv' USING PigStorage(',')

```

```
AS (runner_id:int, checkpoint:int, time:chararray, cups:int);

```

```

-- Join datasets to combine participant info with hydration data
joined_data = JOIN participants BY runner_id, hydration BY runner_id;

-- Filter for runners who took more than 2 cups at any station
hydrated_runners = FILTER joined_data BY hydration::cups > 2;

-- Group by runner_id to calculate total cups consumed per runner
grouped_data = GROUP hydrated_runners BY participants::runner_id;
total_cups = FOREACH grouped_data GENERATE
    group AS runner_id,
    SUM(hydrated_runners.hydration::cups) AS total_cups;

-- Order runners by total cups consumed in descending order
ordered_data = ORDER total_cups BY total_cups DESC;

-- Store the final results back into HDFS
STORE ordered_data INTO 'hdfs://path/to/output/total_cups_per_runner'
    USING PigStorage(';');

-- to verify in Pig shell
-- DUMP ordered_data;

```

This response covers all parts of Question 6 (a-d) and the required four execution steps based on the **Runner Health and Safety** scenario in the test paper.

Question 6: Pipeline Implementation

a) Complete docker-compose.yml File

This file defines the full big data environment, including HDFS, KRaft Kafka, PostgreSQL (using the credentials sama_user/sama_pass from Step 3b), and Superset.

version: '3.8'

services:

1. PostgreSQL (Data Warehouse)

postgres:

image: postgres:14-alpine

container_name: postgres

environment:

- POSTGRES_USER=sama_user**
- POSTGRES_PASSWORD=sama_pass**
- POSTGRES_DB=sama_db**

ports:

- "5432:5432"**

volumes:

- pgdata:/var/lib/postgresql/data**

networks:

- sama_net**

2. HDFS NameNode

namenode:

image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8

container_name: namenode

ports:

- "9870:9870" # HDFS UI**

environment:

- CLUSTER_NAME=comrades_cluster**

volumes:

- hdfs_namenode:/hadoop/dfs/name**

networks:

- sama_net**

3. HDFS DataNode

datanode:

image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8

container_name: datanode

environment:

- SERVICE_PRECONDITION=namenode:9870

volumes:

- hdfs_datanode:/hadoop/dfs/data

networks:

- sama_net

depends_on:

- namenode

4. KRaft Kafka Broker/Controller

kafka:

image: bitnami/kafka:latest

container_name: kafka

ports:

- "9092:9092"

environment:

- KAFKA_ENABLE_KRAFT=yes
- KAFKA_CFG_PROCESS_ROLES=broker,controller
- KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
- KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
-

KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT

- KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092 # Use service name 'kafka'

for inter-container

- KAFKA_BROKER_ID=1
- KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
- ALLOW_PLAINTEXT_LISTENER=yes

volumes:

- kafka_data:/bitnami/kafka

networks:

- sama_net

5. Apache Superset (Visualization)

superset:

image: apache/superset:latest

container_name: superset

ports:

- "8088:8088"

environment:

- SUPERSET_SECRET_KEY=a_secure_secret_key

volumes:

- superset_home:/app/superset_home

networks:

- sama_net

depends_on:

- postgres

volumes:

pgdata:
kafka_data:
hdfs_namenode:
hdfs_datanode:
superset_home:

networks:

sama_net:
driver: bridge

b) Complete schema.sql File

The schema is normalized to handle participant details, real-time sensor data, and medical incident reports.

-- 1. Table for Runner Demographic and Registration Information

```
CREATE TABLE Runners (  
  runner_id INT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  age INT,  
  gender CHAR(1), -- 'M' or 'F'  
  qualifying_time INTERVAL,  
  race_start_time TIMESTAMP NOT NULL  
);
```

-- 2. Table for Real-time Biometric Sensor Data (BiometricData)

-- This captures the high-velocity, high-volume sensor streams.

```
CREATE TABLE BiometricData (  
  log_id SERIAL PRIMARY KEY,  
  runner_id INT REFERENCES Runners(runner_id),  
  timestamp TIMESTAMP NOT NULL,  
  heart_rate INT,      -- Beats Per Minute (BPM)  
  body_temp NUMERIC(4, 2), -- Body Temperature in Celsius  
  battery_level NUMERIC(3, 2), -- Sensor battery percentage  
  CHECK (heart_rate > 0)  
);
```

-- 3. Table for Incident and Medical Station Reports (MedicalIncidents)

-- This captures semi-structured reports logged by medical staff.

```
CREATE TABLE MedicalIncidents (  
  --
```

```

incident_id SERIAL PRIMARY KEY,
runner_id INT REFERENCES Runners(runner_id),
checkpoint_id INT,
timestamp TIMESTAMP NOT NULL,
incident_type VARCHAR(100), -- e.g., Dehydration, Hypoglycemia, Collapse
treatment_provided TEXT, -- Detailed notes on first aid/treatment
status VARCHAR(50) -- e.g., Treated, DNF (Did Not Finish), Hospitalized
);

```

c) Complete producer.py Script

This script simulates sending real-time biometric data to the Kafka topic biometric_stream.

```

import json
import time
import random
from kafka import KafkaProducer

# Configure connection to the Kafka service defined in docker-compose (name is 'kafka')
# Note: In Python outside the container, use 'localhost:9092' if ports are mapped.
producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

TOPIC_NAME = 'biometric_stream'
RUNNER_IDS = [101, 105, 120, 150, 180] # Sample Runner IDs

print(f"Starting producer for topic: {TOPIC_NAME}")

try:
    while True:
        runner = random.choice(RUNNER_IDS)

        # Simulate biometric data: normal HR 150-170, normal temp 37.0-37.5
        hr_base = random.randint(150, 170)
        temp_base = round(random.uniform(37.0, 37.5), 2)

        # Introduce a high-risk spike for a specific runner (e.g., Runner 120)
        if runner == 120 and random.random() < 0.2:
            hr_base = random.randint(185, 200) # Dangerously high HR
            temp_base = round(random.uniform(38.5, 39.5), 2) # High Fever

        message = {
            'runner_id': runner,
            'timestamp': time.time(),

```

```

        'heart_rate': hr_base,
        'body_temp': temp_base,
        'location': f"CP-{random.randint(1, 10)}"
    }

    producer.send(TOPIC_NAME, value=message)
    print(f"Sent: Runner {runner}, HR: {message['heart_rate']], Temp:
{message['body_temp']}")

    time.sleep(0.5) # Send every 0.5 seconds

except KeyboardInterrupt:
    print("\nStopping producer.")
finally:
    producer.flush()
    producer.close()
    print("Producer closed.")

```

d) Complete consumer.py Script

This script simulates the ingestion service, reading real-time biometric data and printing it before it would be written to HDFS/PostgreSQL.

```

import json
from kafka import KafkaConsumer

TOPIC_NAME = 'biometric_stream'

consumer = KafkaConsumer(
    TOPIC_NAME,
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    group_id='health-safety-pipeline-v1',
    value_deserializer=lambda v: json.loads(v.decode('utf-8'))
)

print(f"Starting consumer for topic: {TOPIC_NAME}")
print("Listening for Biometric Data streams...")

try:
    # Continuously listen for messages
    for message in consumer:
        data = message.value

        # Check for immediate high-risk conditions

```

```

if data.get('heart_rate', 0) > 180 and data.get('body_temp', 0) > 38.0:
    print(" 🚨 HIGH ALERT: Runner {} - HR: {} Temp: {} 🚨 ".format(
        data['runner_id'], data['heart_rate'], data['body_temp']))
else:
    print(f"Received: Runner {data['runner_id']} at {data['location']} | HR:
{data['heart_rate']}")

except KeyboardInterrupt:
    print("\nStopping consumer.")
finally:
    consumer.close()
    print("Consumer closed.")

```

Execution Steps (1-4)

These steps execute the pipeline defined in the test paper and the files above.

Step 1: Run the services

The full docker-compose.yml file must be executed to launch the entire pipeline (Postgres, Kafka, HDFS, and Superset).

Action	Command
Start Services	<code>docker compose up -d</code>

Step 2: Initialize the Superset database

Once the Superset container is running (check with `docker ps`), the database must be initialized, and an administrator user created to access the UI.

Action	Command
Create Admin User	<code>docker exec -it superset superset fab create-admin --username admin --firstname Comrades --lastname Admin --email admin@comrades.org --password admin</code>
Upgrade DB	<code>docker exec -it superset superset db upgrade</code>
Initialize Superset	<code>docker exec -it superset superset init</code>

Step 3: Connect Superset to PostgreSQL

The Superset application (accessible at `http://localhost:8088`) must be configured to connect to the internal PostgreSQL service.

Action	Details
Connection String	<code>postgresql://sama_user:sama_pass@postgres:5432/sama_db</code>

Action	Details
Steps	1. Go to Settings $\rightarrow$ Database Connections $\rightarrow$ + Database. 2. Choose PostgreSQL. 3. Paste the connection string. 4. Click Test Connection $\rightarrow$ Save.

Step 4: Create Dashboard and Charts

The final step is to build the visualizations using the processed data from the database.

Action	Details
Dataset Selection	Go to Datasets $\rightarrow$ + Dataset $\rightarrow$ Select PostgreSQL $\rightarrow$ Choose hydration_events table (as per test document).
Chart Creation	i: Bar Chart $\rightarrow$ "Average Cups per Checkpoint" ii: Line Chart $\rightarrow$ "Hydration Trends over Time" iii: Pie Chart $\rightarrow$ "Incidents by Gender"
Dashboard Name	Combine charts into a single dashboard called "Comrades Health & Hydration".

Question 1

a) Formulate a problem statement related to runner health and safety during the marathon [3 marks]

How can the Comrades Marathon organizers leverage real-time biometric data and historical incident reports to predict and proactively mitigate the risk of critical medical incidents (e.g., severe dehydration, heat stroke, cardiac events) among runners, thereby minimizing emergency response times and improving overall race safety?

b) Write two questions that you would ask your data for solving the problem you described in part a) [2 marks]

1. What is the statistical correlation between specific biometric thresholds (e.g., sustained heart rate above 180 BPM coupled with a body temperature above 38.5°C) and the subsequent logging of a critical medical incident within the next 30 minutes?
2. Which sections of the course (checkpoints) and what specific time windows experience the highest concentration of high-risk biometric readings or logged medical incidents, allowing for targeted deployment of medical staff?

Question 2

Complete the table analyzing the two datasets based on the Runner Health and Safety scenario.

Dataset	Data Type	Source	Storage Format (in HDFS)	How it can improve medical response / runner safety
Real-time biometric and health	Semi-structured	Smart wearables (watches, HR	JSON or AVRO (for streaming efficiency) or	Enables real-time risk scoring for individual runners. Organizers can

Dataset	Data Type	Source	Storage Format (in HDFS)	How it can improve medical response / runner safety
sensor data		monitors), timing chips (for location).	Parquet (for analytical speed).	create a prediction model to flag a runner immediately when their vital signs cross pre-defined critical thresholds, allowing medical teams to intercept the runner before a collapse.
Incident and medical station reports	Semi-structured or Unstructured	Medical staff logs (tablets/apps), post-race emergency services documentation.	JSON (structured logs) or TEXT files (narrative notes).	Provides ground truth data to train prediction models. Analysis can identify common injury patterns (e.g., dehydration spikes at specific checkpoints) and confirm which high-risk biometric signals actually resulted in an incident, improving future predictive accuracy.

Question 3

a) Complete the missing parts (see the TODO parts) [8 marks]

The goal is to analyze the average heart rate and body temperature of runners who did not finish (DNF) the race.

Code snippet

-- Load biometric data

```
biometrics = LOAD 'hdfs://data/raw/biometric_data.csv' USING PigStorage(',')
              AS (runner_id:int, timestamp:chararray, heart_rate:int, body_temp:float);
```

-- Load incident data

```
incidents = LOAD 'hdfs://data/raw/medical_incidents.csv' USING PigStorage(',')
            AS (incident_id:int, runner_id:int, status:chararray);
```

-- TODO: Filter the incidents for runners who did not finish (DNF)

```
dnf_incidents = FILTER incidents BY status == 'DNF';
```

-- Join the DNF incidents with the biometric data

```
joined_data = JOIN biometrics BY runner_id, dnf_incidents BY runner_id;
```

-- TODO: Group the joined data by runner_id

```
grouped_dnf_runners = GROUP joined_data BY biometrics::runner_id;
```

-- TODO: Calculate the average heart rate and average body temperature for each DNF runner

```
avg_vitals_dnf = FOREACH grouped_dnf_runners
    GENERATE
        group AS runner_id,
        AVG(joined_data::heart_rate) AS avg_dnf_hr,
        AVG(joined_data::body_temp) AS avg_dnf_temp;
```

-- TODO: Order the results by average body temperature in descending order

```
ordered_by_temp = ORDER avg_vitals_dnf BY avg_dnf_temp DESC;
```

b) Suggest two additional analyses that could be performed on the biometric dataset using Pig [2 marks]

1. Identify High-Variance Runners: Group the biometrics data by runner_id and calculate the Standard Deviation (STDDEV) of the heart_rate. Runners with a very high variance might indicate faulty sensors or erratic performance.
2. Calculate Time at Risk: Filter biometrics for records where heart_rate > 180 and body_temp > 38.0. Group the resulting data by runner_id and use COUNT() to determine how many readings (and thus, roughly how much time) each runner spent in a high-risk zone.

Question 4

a) Write down all the issues in the code [6 marks]

1. Normalization Violation (Data Redundancy): Runner information (name, age, gender) is repeated for every biometric log (runner_id 101 is listed twice). This should be split into a Runners table and a BiometricLogs table.
2. Invalid Column Name: The column body temp contains a space, which is invalid in SQL without being quoted (e.g., "body temp"). It should be body_temp.
3. Syntax Error (Missing Semicolons): The CREATE TABLE and the INSERT INTO statements are both missing their terminating semicolons (;).
4. Syntax Error (Missing Quote): In the third VALUES tuple, the value 'F' is missing its opening single quote.
5. Syntax Error (Missing Value): In the last VALUES tuple, (102, 'Jane D.', 29, 'F', 185,), a value for the body_temp column is missing after the last comma.
6. Inefficient Data Type: Using TEXT for gender is inefficient. A CHAR(1) or a specific ENUM type is better for single-character codes.

b) Write a corrected version of the code [3 marks]

This corrected version implements a normalized schema (two tables) and fixes all syntax errors.

SQL

```
-- 1. Table for Runner Demographic Information (Normalized)
CREATE TABLE Runners (
    runner_id INT PRIMARY KEY,
    name TEXT NOT NULL,
    age INT,
```

```
gender CHAR(1) -- More efficient
);
```

-- 2. Table for Biometric Logs (Normalized)

```
CREATE TABLE BiometricLogs (
  log_id SERIAL PRIMARY KEY,
  runner_id INT REFERENCES Runners(runner_id), -- Foreign Key
  heart_rate INT,
  body_temp NUMERIC(4, 2) -- Corrected column name and used a numeric type
);
```

-- Insert unique runners

```
INSERT INTO Runners (runner_id, name, age, gender)
VALUES
  (101, 'Thabo M.', 35, 'M'),
  (102, 'Jane D.', 29, 'F'); -- Fixed missing quote
```

-- Insert biometric events

```
INSERT INTO BiometricLogs (runner_id, heart_rate, body_temp)
VALUES
  (101, 165, 37.1),
  (101, 170, 37.5),
  (102, 185, 38.2); -- Fixed missing value (assuming 38.2)
```

Question 5

These queries assume you are using the corrected (normalized) database schema (Runners and BiometricLogs) from Question 4b, and that a MedicalIncidents table (as defined in Q6b) also exists.

a) To count the number of high-risk heart rate readings (over 180 BPM) per runner [3 marks]
This query counts how many times each runner's heart rate exceeded the threshold.

SQL

```
SELECT
  r.name,
  COUNT(b.log_id) AS num_high_hr_readings
FROM
  Runners r
JOIN
  BiometricLogs b ON r.runner_id = b.runner_id
WHERE
  b.heart_rate > 180
GROUP BY
  r.name
ORDER BY
  num_high_hr_readings DESC;
```

b) To find runners with a maximum body temperature above 38.5°C AND who have at least one logged medical incident [4 marks]

This query identifies runners who met both the severe temperature threshold and were documented by the medical team.

SQL

```
SELECT
    r.runner_id,
    r.name,
    MAX(b.body_temp) AS max_body_temp
FROM
    Runners r
JOIN
    BiometricLogs b ON r.runner_id = b.runner_id
WHERE
    r.runner_id IN (SELECT DISTINCT runner_id FROM MedicalIncidents) -- Subquery finds runners
with incidents
GROUP BY
    r.runner_id, r.name
HAVING
    MAX(b.body_temp) > 38.5
ORDER BY
    max_body_temp DESC;
```